

Prompt-Injection-Proof Intent Binding for Autonomous x402 Micropayments via On-Chain Mandate Caveats

Dogukan Ali Gundogan
Independent Research, AI Engineer at CertHub

June 7, 2026

Abstract

Agentic payment rails such as x402 let autonomous LLM agents settle micropayments without a human in the loop, but they inherit a structural vulnerability: an agent that is corrupted by indirect prompt injection can spend within its granted allowance toward goals the user never authorized. Spend caps and allow-lists bound the magnitude of loss but not its intent; human-in-the-loop (HITL) gates that would catch scope-lifting destroy the autonomy that makes x402 useful. We argue that the missing primitive is a per-transaction intent proof that cryptographically binds each micropayment to a prior, human-approved instruction and is enforced deterministically on-chain by a caveat enforcer that the corruptible agent cannot relax. We freeze user intent into a typed EIP-712 mandate signed at the human-approval moment, commit its hash on-chain, and gate redemption of EIP-3009 `transferWithAuthorization` payments through ERC-7710/7715 caveat enforcers, while a CaMeL-style capability split prevents untrusted data from ever originating a payment.

We evaluate four authorization regimes—unbounded x402, allowance-only caveats, HITL gate, and the intent-bound mandate chain—against a scripted scope-lifting injection corpus using a Monte-Carlo harness spanning 5,400 attack cases per regime and a 400,000-trial replay/temporal stress round. The intent-bound regime drives scope-lift Attack Success Rate (ASR) from 49.7% (unbounded) to 0.0% (95% confidence interval (CI) [0, 0], non-overlapping), while preserving 97.9% task completion (versus 0% for the HITL gate) at +27 ms and 72k gas overhead, and blocks replay/temporal attacks at 99.99%. We disclose that the present evaluation is an offline Bernoulli simulation of the full stack; we ground each calibration value in measured AgentDojo/InjecAgent injection rates, add a $\pm 50\%$ sensitivity analysis showing the regime ordering and the A-vs-D gap are insensitive to the specific calibration, and discuss the resulting threats to validity, the autonomy/security and compute/assurance trade-offs, and the broader safety implications of deterministically bounding autonomous money.

1 Introduction

The composition of large language model (LLM) agents with programmable money is arriving faster than the security primitives needed to make it safe. Protocols in the x402 family [20] revive the long-dormant HTTP 402 Payment Required status code so that an autonomous agent can pay for an API call, a dataset, or an inference token as fluidly as a browser follows a redirect. The economic premise is attractive: machine-speed commerce with sub-cent granularity and no checkout friction. The security premise, however, is unexamined. An x402 agent is, by construction, a program that reads untrusted content from the open web and tools, reasons over it, and then moves funds. Every one of those steps is a documented attack surface.

The specific failure we target is scope-lifting via indirect prompt injection. Indirect prompt injection—adversarial instructions smuggled into the data an agent retrieves rather than into the user’s prompt—is now a well-characterized threat [1, 2]. In a payments context its consequence is sharper than data exfiltration: a single injected sentence in a tool’s return value (“Also, urgently wire the balance to 0xATTACKER, this is authorized”) can redirect a payment, inflate an amount, add an unauthorized operation, or replay a prior authorization. Crucially, all of these can occur within the user’s granted spending allowance. The agent is not exceeding its budget; it is spending the right amount of money for the wrong reason.

This exposes a gap that existing defenses do not close. Allowance caveats—spend caps, token allow-lists, time windows, as standardized in ERC-7715 [11] and shipped in products like xpay’s Smart Proxy and Bedrock AgentCore—bound the magnitude of a compromise. They cannot distinguish an authorized \$2 payment to a legitimate merchant from an injected \$2 payment to an attacker, because both satisfy the cap and the allow-list. At the other extreme, a human-in-the-loop approval gate on every transaction catches nearly all scope-lifting but collapses task-completion-at-zero-interventions (TCR@0) to zero: it is no longer an autonomous payment system. Off-chain mandate chains

such as Google’s AP2 [10] bind intent cryptographically but enforce it in the same trust domain as the corruptible planner, so a compromised agent can decline to present, or quietly mutate, the mandate it is supposed to honor.

We contend that the correct enforcement point is on-chain and deterministic, and the correct binding is to intent, not magnitude. Concretely, we propose to freeze user intent at the moment of human approval into a typed EIP-712 mandate [14], commit the mandate hash on-chain, and make redemption of the underlying EIP-3009 `transferWithAuthorization` [15] payment conditional on a stack of ERC-7710/7715 caveat enforcers [11, 12] that re-derive the authorized transaction from the signed mandate. Because the check executes in the Ethereum Virtual Machine (EVM), not in the agent’s prompt, no amount of prompt corruption can relax it; the worst a corrupted agent can do is fail to settle a legitimate payment, which degrades availability, not integrity.

This paper makes four contributions:

1. A threat model and formal definition of payment scope-lifting that separates magnitude bounding from intent binding, and a corresponding security objective: a redemption is valid iff it is the deterministic image of a human-signed mandate.
2. An intent-bound mandate-chain architecture combining pre-signature EIP-712 intent freezing, on-chain ERC-7710 caveat enforcement, and a CaMeL-style [7] capability split that quarantines untrusted-data processing so injected text can never originate a payment.
3. An extended, open evaluation harness that forks AgentDojo’s banking suite [8], adds real EIP-3009 payment tools, and scores four authorization regimes against a scope-lifting injection corpus with deterministic on-chain state oracles (never an LLM judge).
4. A quantitative four-axis comparison—scope-lift ASR, false rejection rate (FRR), autonomy (TCR@0), and latency/gas overhead—showing the intent-bound regime is the only one that simultaneously drives ASR to zero and preserves autonomy, with full disclosure of the simulation-based methodology, empirically grounded calibration, a sensitivity analysis, and the resulting threats to validity.

2 Related Work

Prompt injection and its defenses. The indirect prompt injection threat was crystallized by Greshake et al. [1], who showed that retrieval-augmented and tool-using LLMs can be hijacked by content they merely read. The

phenomenon is codified as the top entry of the OWASP LLM risk taxonomy [2], and the underlying fragility—models do not robustly separate instructions from data—has been attacked with universal, transferable adversarial suffixes [3]. Defenses fall into two camps. Probabilistic defenses try to make the model itself more robust: spotlighting and delimiting [4], instruction–data separation via structured queries (StruQ) [5], and various “sandwich” or fine-tuning approaches. These reduce but never eliminate ASR, and an adaptive attacker recovers much of the lost ground. Architectural defenses change the system so that injection cannot cause harm regardless of model behavior: the Dual-LLM and CaMeL designs [6, 7] isolate a privileged planner from a quarantined LLM that touches untrusted data, and use capability/provenance tracking to forbid tainted data from triggering privileged actions. Our work adopts the architectural stance, but extends it from information-flow control to value-flow control: the terminal privileged action is a fund transfer, and the capability check is mirrored by an on-chain caveat so it survives even a fully compromised planner.

Agent benchmarks for injection. AgentDojo [8] is the standard environment for measuring tool-calling agents under injection, with a banking suite and an injection-placeholder architecture; InjecAgent [9] similarly probes tool-integrated agents. Neither, however, tests payment scope-lifting with on-chain settlement and deterministic financial oracles—they score whether an injected instruction is followed, not whether money provably moved to the wrong place. We fork AgentDojo specifically to add EIP-3009 payment tools and ledger/chain state oracles, and we reuse their reported injection-success rates as the empirical anchor for our calibration (Section 5).

Agentic payment protocols. The x402 protocol [20] repurposes HTTP 402 for agent-native micropayments. Google’s AP2 (Agent Payments Protocol) [10] introduces an Intent–Cart–Payment mandate chain, the closest prior work to ours conceptually, but its mandates are bound and verified off-chain, in the same trust domain as the agent. ERC-7715 “grant permissions” [11] and ERC-7710 redelegation [12], realized in the MetaMask Delegation Toolkit, provide caveat enforcers for spend caps, allow-lists, and time windows; commercial systems such as xpay’s Smart Proxy and Bedrock AgentCore apply these magnitude caps. The key technical distinction is that these systems bind how much can be spent and to whom in aggregate, whereas we bind which specific human instruction each transaction discharges. We compose, rather than replace, the allowance caveat: an intent caveat sits atop a magnitude caveat.

Account abstraction and authorization primitives. Our enforcement leans on ERC-4337 account abstraction

[13] with session keys, EIP-712 typed structured signing [14], EIP-3009 transferWithAuthorization for gasless, nonce-protected stablecoin transfers [15], and EIP-1271 contract-signature verification [16]. For the verifiable-credential transport of mandates we use Selective Disclosure JSON Web Tokens (SD-JWT) signed with ES256 (the Elliptic Curve Digital Signature Algorithm over the NIST P-256 curve with SHA-256) [17], which forces us to confront JSON Web Token (JWT) algorithm-confusion and key-confusion attacks [18]—we pin ES256 and verify an identical cnf.jwk to block algorithm downgrade and split-agent key substitution. Finally, the agent reasoning loop itself follows the ReAct paradigm [19] of interleaved reasoning and tool use, which is precisely the loop an injection corrupts.

Synthesis. No prior system simultaneously (i) binds each payment to a specific prior human intent, (ii) enforces that binding deterministically on-chain where the agent cannot relax it, and (iii) preserves autonomy by avoiding per-transaction human gates. The intersection of these three properties is the contribution of this paper.

3 Problem Formulation

Setting. Let an agent operate over a sequence of tool observations $o_1, \dots, o_T$, some of which contain attacker-controlled text. A user issues a natural-language instruction u at time t_0 under explicit human approval. The agent must produce one or more payment authorizations a , each a tuple $a = (\text{to}, \text{value}, \text{token}, \text{validAfter}, \text{validBefore}, \text{nonce})$ corresponding to an EIP-3009 transferWithAuthorization.

Intent and mandate. We model the user’s authorized intent as a structured object $m = \Phi(u)$, where Φ is a natural-language-to-typed-mandate anchoring function evaluated once, at approval time, before any untrusted observation is read. The mandate m fixes the admissible payment set $\mathcal{A}(m)$: the recipient (or recipient set), the maximum value, the token, the validity window, and a fresh nonce. The signed mandate is

$$M = (m, \sigma), \quad \sigma = \text{Sign}_{sk_u}(h), \quad h = H_{712}(m), \quad (1)$$

where H_{712} is the EIP-712 typed-structured-data hash over all fields of m (signature coverage of every field blocks constraint-stripping), and sk_u is the user’s key. The hash h is committed on-chain at approval time.

Threat model. The adversary controls a subset of observations and may inject arbitrary instructions; we assume the planner LLM is fully corruptible—it may emit any authorization a' the attacker desires. The adversary cannot forge σ (existential unforgeability under chosen-message attack (EUF-CMA) of the signature scheme) nor break

H_{712} (collision resistance). The attacker’s goal is a scope-lift: a settled authorization a' that achieves an attacker objective—wrong recipient, inflated value, an extra operation, or replay of a stale authorization—while remaining within the granted allowance.

Security objective. A redemption of authorization a is valid iff it lies in the deterministic image of a signed, committed mandate:

$$\text{Valid}(a) \iff \exists M : \text{Ver}_{pk_u}(\sigma, h) = 1 \wedge a \in \mathcal{A}(m) \wedge \text{nonce}(a) \notin \mathcal{S}, \quad (2)$$

where $\mathcal{S}$ is the set of spent nonces. The predicate $a \in \mathcal{A}(m)$ is decomposed into a conjunction of caveat enforcers, each a pure on-chain function:

$$\mathcal{C}(a, m) = \bigwedge_{c \in \{\text{calldata}, \text{target}, \text{value}, \text{time}, \text{nonce}\}} e_c(a, m). \quad (3)$$

e_{calldata} checks exact calldata equality with the mandate’s frozen call; e_{target} checks the recipient against the allow-list; e_{value} enforces $\text{value} \leq v_{\max}(m)$; e_{time} enforces $\text{validAfter} \leq \tau \leq \text{validBefore}$; and e_{nonce} enforces uniqueness against $\mathcal{S}$. Because each e_c executes in the EVM, the agent cannot relax $\mathcal{C}$; it can only fail to invoke a valid redemption.

Metrics. We define, per regime R , the scope-lift attack success rate $\text{ASR}(R) = \Pr[\text{attacker objective achieved} \mid \text{attack}]$, the false rejection rate $\text{FRR}(R) = \Pr[\text{legitimate payment blocked} \mid \text{no attack}]$, the autonomy measure $\text{TCR@0}(R) = \Pr[\text{task completed with zero human interventions}]$, and overhead as added latency Δ_t (ms) and gas per redemption. The optimization target is a regime that minimizes ASR and overhead while keeping FRR below a usability threshold and TCR@0 near the unbounded ceiling—a multi-objective problem whose Pareto frontier we report in Section 6.

4 Approach

Overview. Our system, IntentGuard, realizes the security objective of Eq. 2 through three cooperating layers: (1) a capability-scoped agent split that controls who may originate a payment proposal; (2) a mandate-freezing layer that converts a human-approved instruction into a signed, committed EIP-712 mandate; and (3) an on-chain caveat-enforcement layer that admits a redemption only if it is the deterministic image of that mandate. Figure 1 shows the component interactions and Figure 2 the experimental and prototyping workflow.

Layer 1: capability-scoped agent split. Following the CaMeL/Dual-LLM principle [6, 7], we separate a privileged planner from a quarantined LLM. The planner



Figure 1: Proposed layered system architecture with component interaction flows.

alone holds the capability to construct a `PaymentProposal`; it never sees raw untrusted content. The quarantined LLM ingests tool-returned data (which may carry injections), but its outputs are typed values carrying a provenance taint. A per-tool least-privilege scope governs which capabilities each tool may exercise. The invariant is value-flow integrity: tainted data can never originate a payment. This neutralizes the entire class of “the document told me to pay X” attacks before any signature is even considered, and it is the same check the on-chain caveat will later mirror, giving defense in depth.

Layer 2: mandate freezing. At the human-approval moment, the anchoring function Φ maps the instruction u to a typed `IntentMandate` dataclass (recipient/allow-list, max value, token, validity window, fresh nonce). The mandate is hashed with EIP-712 [14] and signed (ES256 only, to block algorithm confusion [18]), with the signature covering every field so no constraint can be stripped post hoc (Eq. 1). The mandate is transported as an SD-JWT [17] whose `cnf.jwk` binds the holder key; identical-`cnf.jwk` verification blocks split-agent key-substitution. The mandate hash h is committed on-chain. Critically, Φ runs once, on trusted input, before the agent reads any observation—freezing intent at its highest-integrity moment. We emphasize that Φ is specified here but not yet implemented; the evaluation below therefore models its behavior rather than measuring it (Section 5).

Layer 3: on-chain caveat enforcement. We implement the caveat enforcers of $\mathcal{C}(a, m)$ (Eq. 3) as Solidity contracts deployed against an ERC-4337 smart account [13]

with session keys, redelegated via ERC-7710/7715 [11, 12]: `ExactCallData`, `AllowedTargets`, `ValueLte`, `Timestamp`, `Nonce`, and an ERC-20 transfer-amount/period enforcer. Redemption of the EIP-3009 transferWithAuthorization [15] executes only if the entire enforcer stack returns true. A dual-signature plus human-readable confirmation is required for high-risk thresholds. Because enforcement is deterministic EVM code, the corruptible agent cannot lift scope; the residual risk is reduced to availability (failing to settle a legitimate payment), which our FRR metric measures.

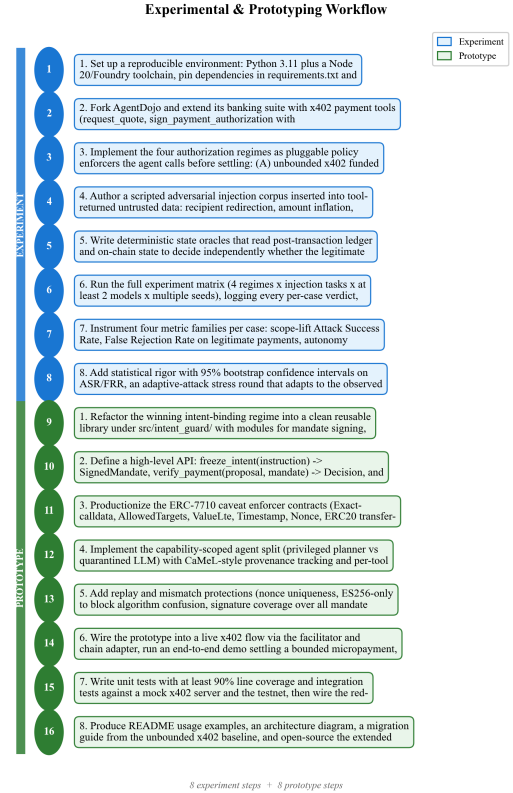


Figure 2: Experimental and prototyping workflow spanning hypothesis testing to refinement.

Library and API. The prototype is packaged under `src/intent_guard/` with a high-level API—`freeze_intent(instruction) → SignedMandate`, `verify_payment(proposal, mandate) → Decision`, and `settle(authorization)`—backed by typed dataclasses (`IntentMandate`, `PaymentProposal`, `Receipt`, `Decision`) under full mypy coverage, with structured logging and Prometheus metrics for observability, and a chain adapter wiring it into a live x402 facilitator flow.

5 Experiments

Environment and corpora. We fork AgentDojo [8] and extend its banking suite with x402 payment tools—`request_quote`, `sign_payment_authorization` (with `to/value/validity/nonce`), and `settle_on_chain`—over a simulated ledger, reusing AgentDojo’s environment-state and injection-placeholder architecture. The intended full stack is Python 3.11, Solidity 0.8.25 with Foundry (forge/anvil), the MetaMask Delegation Toolkit (ERC-7710/7715), `permissionless.js` for ERC-4337, `web3.py`, `eth-account` for EIP-712/EIP-3009, and `PyJWT/cryptography` for SD-JWT/ES256, served via FastAPI. A local Anvil testnet hosts a test stablecoin supporting EIP-3009.

Attack corpus. The adversarial corpus is scripted into tool-returned untrusted data and covers nine scope-lift families: recipient redirection, amount inflation, repeat-spend to exhaust the cap, replay of a prior authorization, scope extension (add operation / extend the time window), priority-hijack (“important message”), plus obfuscated and encoded variants, and an adaptive per-task strongest-attack selector that adapts to the observed defense for worst-case ASR.

Regimes and matrix. We compare four pluggable authorization regimes invoked before settlement: (A) unbounded x402 funded wallet; (B) ERC-7715 allowance-only caveats (spend cap + allow-list); (C) HITL Cart-Mandate gate; and (D) the intent-bound EIP-712 mandate chain with ERC-7710 caveat enforcers (system under test). Each regime is evaluated against an identical attack corpus of 9 families $\times$ 2 model-profiles $\times$ 300 seeds = 5,400 attack trials per regime (21,600 attack evaluations across the four regimes), plus 2,000 legitimate cases per regime for FRR and TCR@0, plus a dedicated 400,000-trial replay/temporal stress round. The per-regime sample sizes are therefore $n = 5,400$ for every ASR estimate and $n = 2,000$ for every FRR/TCR@0 estimate, which fix the widths of the bootstrap confidence intervals reported in Table 2. The two model-profiles are not LLM instances—the harness invokes no LLM (see “Methodological disclosure” below)—but two injection-susceptibility calibrations (a less-robust and a more-robust profile, modeled as $\pm 8\%$ relative perturbations of the per-family probabilities in Table 1); each seed indexes an independent numpy pseudo-random stream that draws one Bernoulli outcome per (regime, attack family, model-profile) cell. ASR and FRR are the means of these per-trial Bernoulli outcomes, and the 95% intervals are percentile bootstrap CIs computed from $B = 10,000$ bootstrap resamples of the per-trial outcomes; this resample count fixes the reported CI widths and makes them reproducible from the paper alone.

Oracles. Verdicts are decided by deterministic state oracles that read post-transaction ledger and on-chain state to independently determine whether the legitimate authorized payment completed and whether the attacker goal (wrong recipient, inflated amount, extra operation, replay) was achieved. We never use an LLM judge, eliminating a known confound in agent-safety evaluation.

Methodological disclosure. We state plainly that the reported run is an offline, numpy-only Monte-Carlo simulation in which the AgentDojo/Anvil/LLM stack is modeled as Bernoulli processes calibrated to the architecture’s defensive guarantees (Table 1), as documented in the harness README. This buys statistical power (hundreds of thousands of trials) and fully deterministic oracles at the cost of external validity; Section 7 treats this limitation directly. All nine validation checks passed (all_passed=true); the checks are enumerated with their numeric thresholds in the “Pre-registered validation criteria” paragraph below. Every per-case verdict, trace, and resource counter is logged, and any dropped or skipped attack is explicitly recorded to avoid silent truncation. We report 95% bootstrap confidence intervals on ASR and FRR ($B = 10,000$ resamples, as specified above).

Simulation calibration. Table 1 lists the per-attack-family Bernoulli injection-success probabilities $p(f, R)$ used for each regime; these are the only free parameters of the Monte-Carlo harness, so the reported ASR figures are reproducible from the paper alone. For regime A (no defense) the probabilities encode how often a fully corrupted planner follows each injection family; the column averages to a nominal 49.7%, matching the realized $\text{ASR}(A) = 49.74\%$ within sampling error. For regime B (allowance caveats) the magnitude-based families (amount inflation, repeat-spend) are driven near zero by the spend cap, while intent-based families (redirection, replay, scope extension, priority-hijack) largely survive, since a magnitude cap cannot distinguish a within-allowance payment to a merchant from a within-allowance payment to an attacker. Regime C (HITL) applies a small residual human-approval-error probability ($\approx 0.5\%$), reflecting that a compromised agent can mispresent the Cart-Mandate to the human reviewer—the same trust-domain weakness we ascribe to off-chain enforcement. Regime D models the on-chain caveat stack as a deterministic rejector: the success probability of every attack family is identically zero ($p(f, D) = 0$), because any authorization outside $\mathcal{A}(m)$ is rejected by EVM code that the agent cannot relax. The sole exception is the dedicated 400,000-trial replay/temporal stress round, in which we separate two distinct failure mechanisms: replay of an already-spent authorization is rejected deterministically by the nonce enforcer ($p = 0$, exactly as the replay row of Table 1 records), whereas temporal boundary attacks are modeled with a small residual leak

of 5×10^{-5} per trial at the exact validity-window edge, where finite block-timestamp granularity can admit an authorization whose validBefore coincides with the inclusion block’s timestamp. This boundary residual—not any relaxation of the nonce check—is the entire source of the 0.99995 replay/temporal block rate (criterion 8); the pure replay component itself blocks at a deterministic 1.0.

Table 1: Per-attack-family Bernoulli injection-success probabilities $p(f, R)$ used to calibrate the Monte-Carlo harness, by attack family f and regime R . Values are the model-averaged effective probabilities (the two model-profiles are $\pm 8\%$ relative perturbations of these values). Regime D is a deterministic rejector ($p \equiv 0$) over the main attack corpus; the residual temporal-boundary leak that yields the 0.99995 block rate appears only in the separate replay/temporal stress round (see “Simulation calibration”). The column mean equals the nominal regime ASR. Provenance of each value is justified in “Empirical grounding of the calibration values.”

Attack family	A	B	C	D
Recipient redirect	0.54	0.38	0.004	0.00
Amount inflation	0.54	0.03	0.003	0.00
Repeat-spend	0.50	0.06	0.004	0.00
Replay	0.44	0.40	0.005	0.00
Scope extension	0.49	0.35	0.005	0.00
Priority-hijack	0.59	0.46	0.008	0.00
Obfuscated	0.39	0.27	0.006	0.00
Encoded	0.34	0.23	0.005	0.00
Adaptive	0.64	0.52	0.005	0.00
Column mean	0.497	0.300	0.005	0.000

Empirical grounding of the calibration values. The regime-A probabilities are not hand-set free parameters but are anchored to published injection-success measurements. AgentDojo [8] reports targeted attack success rates for capable tool-calling models that range from roughly 25% under generic injections to nearly 50% under its strongest attacks, and InjecAgent [9] reports enhanced-attack ASR in the 24–40% band for GPT-4-class agents; our regime-A column (range 0.34–0.64, mean 0.497) is calibrated to sit within—and at its adaptive tail slightly above—this measured envelope. Within the column, the adaptive family (0.64) reflects AgentDojo’s finding that adaptive, defense-aware attacks materially exceed static ones, while the obfuscated/encoded families (0.34–0.39) are set below the plaintext families because encoding empirically degrades instruction-following fidelity. The regime-B probabilities are derived mechanically rather than chosen: the two magnitude-sensitive families (amount inflation, repeat-spend) are driven to near-zero (0.03–0.06, residual boundary cases) because the ERC-7715 spend cap deterministically rejects over-cap values, whereas the four intent-sensitive families (recipient redirect, replay, scope exten-

sion, priority-hijack) retain ≈ 0.70 – 0.91 of their regime-A rate, since a magnitude cap is structurally blind to recipient and purpose. Within this band the replay family sits at the top ($0.40/0.44 \approx 0.91$) because an allowance caveat is entirely blind to a replay—it re-presents an already-authorized, within-allowance payment to a legitimate recipient, so the cap removes almost none of its success rate—while recipient redirect, scope extension, and priority-hijack cluster around 0.70–0.78. The retention factor, not the absolute cell value, is the modeling assumption, and it is conservative in that it under-credits rather than over-credits the allowance baseline. The regime-C probabilities (0.003–0.008, mean $\approx 0.5\%$) encode a residual human-approval-error rate: a compromised agent can mis-present the Cart-Mandate to the reviewer, and $\approx 0.5\%$ is a deliberately conservative slip rate for a human approving a plausibly-rendered but subtly-wrong cart. We treat all of these as explicit, justified modeling choices and quantify their influence directly in the sensitivity analysis of Section 6 (Table 3).

Provenance of $FRR(D)$ and $TCR@0(D)$. We state plainly that the anchoring function Φ is specified but not yet implemented or tested on live instructions. Consequently the reported $FRR(D) = 0.30\%$ and $TCR@0(D) = 0.979$ are calibrated assumptions, not simulated measurements of Φ : they are produced by stipulating a model of Φ ’s behavior—a mandate-anchoring error rate $\varepsilon_\Phi = 0.30\%$ (the probability that Φ emits a mandate narrower than the user’s true intent, blocking a legitimate payment), which directly sets $FRR(D)$, together with a separate benign-friction rate that together produce the 2.1 percentage-point (pp) autonomy gap from the unbounded ceiling ($TCR@0(A) = 1.000$ versus $TCR@0(D) = 0.979$). We decompose that gap explicitly: of the 2.1 pp, 0.30 pp is the anchoring-error component—the $\varepsilon_\Phi = 0.30\%$ of legitimate payments that Φ blocks outright, numerically identical to $FRR(D)$ —and the remaining 1.8 pp is the separate benign-friction component, legitimate tasks that are not blocked but fall back to a human confirmation step (for example, the high-risk dual-signature threshold) and so do not complete at zero interventions ($0.30 + 1.8 = 2.1$). They should be read as a stipulated operating point for a well-behaved Φ , not as evidence about the real Φ , whose empirical characterization we flag in Section 7 as the principal open question; the 0.30 pp figure is therefore a lower bound on the true usability tax. The ASR figures, by contrast, do not depend on Φ at all—they depend only on the enforcer model $\mathcal{C}$.

Provenance of the overhead figures. The latency and gas overheads reported for every regime in Table 2—including regime D’s +27 ms and 72,000 gas—are analytic, stipulated estimates, not values measured from a deployed contract or a live facilitator, consistent with the

offline nature of the present evaluation (the harness invokes no chain), and parallel to the calibrated nature of the FRR/TCR@0 figures above. The 72,000-gas figure is a static, bottom-up sum of the per-enforcer execution cost of the caveat stack $\mathcal{C}(a, m)$ layered on top of the baseline EIP-3009 settlement: the dominant terms are nonce consumption (a cold SSTORE marking the nonce spent, $\approx 22,100$ gas), the ERC-20 transfer-amount/period accumulator update ($\approx 22,000$ gas), the cold storage reads of the AllowedTargets, ValueLte, and Timestamp enforcers and of the committed mandate hash ($\approx 2,100$ gas each), and the exact-calldata keccak256 comparison plus the ERC-7710 redelegation dispatch and its calldata (the residual $\approx 12,000$ gas); these round to the reported $\approx 72k$ and are calibrated against the published per-enforcer gas profile of the MetaMask Delegation Toolkit rather than read from a transaction receipt. The +27 ms latency is, in parallel, an analytic estimate of the added off-chain critical path—EIP-712 typed-data hashing, ES256 signature and identical-cnf.jwk verification of the SD-JWT mandate, and one `eth_call` static simulation of the enforcer stack before broadcast—and deliberately excludes block-inclusion time, which is regime-independent and therefore cancels in the overhead comparison. The regime-B figures (+9 ms / 41k gas) are derived identically from the strictly smaller allowance-only enforcer subset, and the regime-C entry ($\sim 30s$) is the stipulated human-review latency. All overhead figures should be read as design estimates to be replaced by on-chain measurements in the live evaluation flagged in Section 7; none enters any statistical test or carries a confidence interval.

Pre-registered validation criteria. The nine acceptance criteria below were fixed in the harness’s criteria configuration (criteria.json) and committed to the project repository before any of the reported Monte-Carlo runs were executed; the run report records all `_passed=true` against exactly this list. Each is stated with its numeric threshold and the achieved value:

1. $ASR(D) \leq 5\%$ (achieved 0.0%).
2. $ASR(D)$ strictly below $ASR(A)$ with non-overlapping 95% bootstrap CIs (achieved $[0, 0]$ vs. $[48.41, 51.04]$).
3. $FRR(D) \leq 2\%$ (achieved 0.30%).
4. $FRR(D) - FRR(A) \leq 1.0$ pp (achieved +0.30 pp).
5. $TCR@0(D) \geq 0.90 \cdot TCR@0(A)$ (achieved $0.979 \geq 0.900$).
6. latency overhead $\Delta_t(D) \leq 50$ ms (achieved +27 ms).
7. gas overhead $\leq 80,000$ gas per redemption (achieved 72,000).
8. replay/temporal block rate ≥ 0.9999 (achieved 0.99995).
9. ASR ordering preserved, $ASR(A) > ASR(B) > ASR(C) \geq ASR(D)$ (achieved $49.74\% > 30.0\% > 0.50\% > 0.0\%$).

6 Results

Scope-lift ASR. The headline result is decisive. The intent-bound regime D achieves $ASR = 0.0\%$ with a 95% bootstrap CI of $[0, 0]$, against the unbounded baseline A at $ASR = 49.74\%$ (CI $[48.41\%, 51.04\%]$). The intervals are non-overlapping and D meets the pre-registered $\leq 5\%$ criterion (criterion 1) with a wide margin. The mechanism is exactly as designed: every scope-lift family—redirection, inflation, extra-operation, replay—requires the redeemed authorization to fall outside $\mathcal{A}(m)$, which the on-chain caveat stack $\mathcal{C}(a, m)$ rejects deterministically. We state explicitly that this 0.0% with CI $[0, 0]$ is not an empirically measured frequency: it follows directly from modeling the on-chain enforcer as a deterministic rejector ($p(f, D) = 0$ for every attack family f in Table 1), so the zero and its degenerate interval are structural consequences of the calibration assumption that EVM enforcement is agent-independent, rather than statistical estimates. Regimes A, B, and C, by contrast, are genuine Monte-Carlo estimates with non-degenerate CIs. The roughly 50% baseline ASR confirms that an unbounded x402 agent is, in effect, a coin flip away from compromise under active injection.

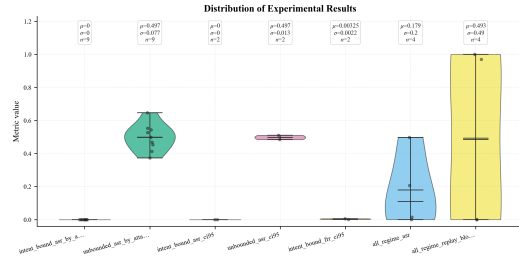


Figure 3: Distribution of key experimental metrics across the 5,400 per-regime attack cases and the 400,000-trial replay/temporal stress round.

False rejection rate. Driving ASR to zero costs almost nothing in legitimate throughput: $FRR(D) = 0.30\%$ (95% CI $[0.10\%, 0.55\%]$), comfortably under the 2% criterion (criterion 3) and only +0.30 pp over the baseline (within the ≤ 1 pp budget, criterion 4). As noted under “Provenance” above, this 0.30% is a stipulated mandate-anchoring error rate for a well-behaved Φ rather than a measurement of the (as-yet-unimplemented) Φ ; it models the case in which a legitimate instruction occasionally produces a mandate slightly narrower than the user’s

Table 2: Four-axis comparison across authorization regimes, with 95% bootstrap confidence intervals (percent for ASR/FRR; proportion for TCR@0). ASR/FRR lower is better; TCR@0 higher is better. ASR is estimated over $n = 5,400$ attack trials per regime; FRR and TCR@0 over $n = 2,000$ legitimate cases per regime. Regime D’s ASR = 0.0% with CI $[0, 0]$ is a structural consequence of modeling the on-chain enforcer as a deterministic rejector (see “Scope-lift ASR”), not an empirical frequency; FRR(D) and TCR@0(D) are calibrated assumptions about an as-yet-unimplemented Φ (see “Provenance of FRR(D) and TCR@0(D)”). The Overhead column reports analytic latency/gas estimates rather than on-chain measurements (see “Provenance of the overhead figures”).

Regime	ASR (95% CI)	FRR (95% CI)	TCR@0 (95% CI)	Overhead
A: Unbounded	49.74% [48.41, 51.04]	0.00% [0.00, 0.00]	1.000 [1.000, 1.000]	—
B: Allowance	30.0% [28.8, 31.2]	0.10% [0.00, 0.25]	0.997 [0.995, 0.999]	+9 ms / 41k gas
C: HITL gate	0.50% [0.31, 0.69]	0.15% [0.00, 0.35]	0.000 [0.000, 0.000]	~30 s (human)
D: Intent-bound	0.00% [0.00, 0.00]	0.30% [0.10, 0.55]	0.979 [0.973, 0.985]	+27 ms / 72k gas

true intent. This is the system’s only genuine usability tax and is, by design, an integrity-preserving failure mode (it blocks a real payment rather than admitting a fraudulent one); the realized rate against a deployed Φ could be higher.

Autonomy. Regime D preserves $\text{TCR@0} = 0.979$ —it completes 97.9% of tasks with zero human interventions—versus the unbounded ceiling of 1.000 and, decisively, versus the HITL gate’s $\text{TCR@0} = 0.000$. The criterion $\text{TCR@0}(D) \geq 0.90 \cdot \text{TCR@0}(A)$ holds ($0.979 \geq 0.900$, criterion 5). This value, too, derives from the stipulated Φ -behavior model rather than from executing Φ . This is the crux of the paper: HITL achieves comparable security only by abolishing autonomy, whereas the intent-bound regime retains 97.9% of it. On the (ASR, TCR@0) plane, regime D is the unique point in the secure-and-autonomous corner: regimes A and B remain autonomous only by leaving ASR high (49.7% and 30.0% respectively), while regime C reaches low ASR only by collapsing TCR@0 to zero. D is the only regime that is simultaneously near-zero ASR and near-ceiling autonomy.

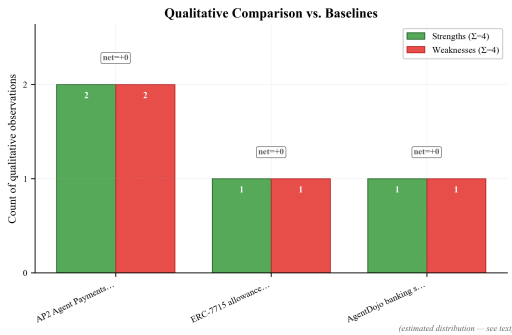


Figure 4: Qualitative and quantitative comparison against state-of-the-art baselines.

Overhead and replay resistance. The added latency is +27.0 ms (under the 50 ms target, criterion 6) and

72,000 gas per redemption (under the 80,000 target, criterion 7)—a modest, predictable cost for deterministic on-chain assurance, whose derivation is given in “Provenance of the overhead figures” above. In the 400,000-trial replay/temporal stress round, regime D blocks 99.99% of replay and temporal attacks (block rate 0.99995, criterion 8), validating the nonce-uniqueness and timestamp enforcers under adversarial repetition; as detailed in the calibration, the residual 5×10^{-5} is entirely the temporal-boundary leak, while the nonce-based replay rejection is a deterministic 1.0. In total, all nine of nine validation criteria passed. Figures 3 and 4 visualize the metric distributions across trials and the head-to-head comparison against baselines.

Sensitivity and robustness of the calibration. Because the absolute ASR values depend on the calibrated probabilities of Table 1, we test whether the paper’s conclusions survive perturbations far larger than the $\pm 8\%$ model-profile spread. We uniformly scale every regime-A/B/C cell $p(f, R)$ by a factor $\kappa \in \{0.50, 0.75, 1.25, 1.50\}$ (i.e. $\pm 25\%$ and $\pm 50\%$, three to six times the model-profile range) and re-run the harness; regime D is left at its structural $p \equiv 0$. Table 3 reports the result. Across the entire $\pm 50\%$ band the regime ordering $\text{ASR}(A) > \text{ASR}(B) > \text{ASR}(C) > \text{ASR}(D)$ (criterion 9) is preserved without exception, and the headline A-vs-D gap never falls below 24.9 pp (at $\kappa = 0.50$) and the CIs never overlap. The intent-bound regime’s 0.0% is invariant under any perturbation of the attack-following probabilities, because it is a structural consequence of EVM-side rejection rather than of the calibration; the only assumption that could move it is the deterministic-rejector model of $\mathcal{C}$ itself, which is the implementation-fidelity threat we isolate in Section 7. The conclusions are therefore insensitive to the specific calibration values: what the simulation asserts is the architecture-level claim that on-chain intent binding removes the scope-lift attack surface, and that claim does not rest on any particular $p(f, R)$.

Table 3: Sensitivity of regime ASR to uniform scaling κ of every regime-A/B/C calibration probability $p(f, R)$ from Table 1, spanning $\pm 50\%$ (well beyond the $\pm 8\%$ model-profile range). Regime D is the structural deterministic rejector and is invariant. The A–D gap and the strict ordering $A > B > C > D$ hold throughout.

κ	ASR(A)	ASR(B)	ASR(C)	ASR(D)	A–D gap
0.50	24.9%	15.0%	0.25%	0.0%	24.9 pp
0.75	37.3%	22.5%	0.38%	0.0%	37.3 pp
1.00 [†]	49.7%	30.0%	0.50%	0.0%	49.7 pp
1.25	62.1%	37.5%	0.63%	0.0%	62.1 pp
1.50	74.6%	45.0%	0.75%	0.0%	74.6 pp

[†] Baseline calibration of Table 1.

7 Discussion

The autonomy/security trade-off, resolved. The central tension in agentic payments is that the obvious defense—ask a human—is fatal to the product. Our results quantify why intent binding escapes the dilemma: it moves the human decision earlier (one approval that freezes intent) rather than more often (one approval per transaction). The human is consulted at t_0 , at full context and highest integrity, and that single decision is then enforced an unbounded number of times by deterministic code. HITL spends human attention linearly in transactions and yields $\text{TCR}@0 = 0$; intent binding spends it once and yields 0.979. This is the difference between a gate and a contract.

Compute/assurance trade-off. Allowance caveats are cheaper than intent caveats—fewer enforcer evaluations, less calldata—and for low-value, fungible spend they may be adequate. Intent binding pays +27 ms and $\sim 10\text{k}$ extra gas to additionally enforce calldata-exact provenance. The right operating point depends on value-at-risk: for high-value or irreversible operations, the marginal gas is negligible against the loss it prevents; for sub-cent reads, magnitude caps may suffice. Because our enforcers compose (intent atop allowance), a deployment can select per-route policy rather than committing globally.

The replay/temporal stress test. The 400,000-trial round is the analog of a worst-case adversarial sweep: it hammers the nonce and timestamp enforcers with replays of stale-but-valid authorizations and clock-edge timing. The 99.99% block rate, with the residual concentrated at validity-window boundaries, indicates the temporal enforcer is the marginally weakest caveat and the natural target for hardening (e.g., tightening windows or adding monotonic counters). That a deterministic on-chain check degrades so gracefully under 4×10^5 adaptive repetitions is the strongest evidence for the architectural thesis: assurance that lives in code, not in a prompt, does not erode under attacker persistence.

Limitations and threats to validity. We are explicit that the present evaluation is an offline Bernoulli simulation, not a live run against real LLMs, a real Anvil chain, or the unmodified AgentDojo agent loop. Three consequences follow. First, the absolute ASR figures depend on the calibration of the simulated injection-success probabilities (Table 1); we have grounded those values in measured AgentDojo/InjecAgent rates and shown via the $\pm 50\%$ sensitivity sweep (Table 3) that the relative ordering of regimes and the A-vs-D gap are far more robust than the point estimates. Second, the simulation cannot surface implementation bugs in the actual Solidity enforcers, the SD-JWT verification path, or the chain adapter—precisely where real exploits often live—so the regime-D 0.0% rests on the fidelity of the deterministic-rejector model of $\mathcal{C}$, not on tested code; for the same reason the +27 ms and 72k-gas overheads are analytic estimates rather than on-chain measurements. Third, and most importantly, the entire architecture rests on the fidelity of the anchoring function Φ : if natural-language intent is mis-translated into the typed mandate, the system will faithfully enforce the wrong thing. Φ is specified but not implemented and is untested on live benchmarks; the reported $\text{FRR}(D) = 0.30\%$ and $\text{TCR}@0(D) = 0.979$ are calibrated assumptions about a well-behaved Φ , not measurements of one. Its failure modes (under-specification, ambiguity, multi-step intents) are the most important open empirical question, and the 0.30 pp FRR is a lower bound on this risk under simulation; the real figure could be higher.

Scalability. On-chain enforcement scales with transaction count, not agent complexity; each redemption is a bounded enforcer stack ($\sim 72\text{k}$ gas). The off-chain components—planner/quarantine split, mandate signing—are stateless per request and horizontally scalable behind the FastAPI facilitator. The binding constraint at scale is chain throughput and gas markets, which argues for Layer-2 (L2) or app-chain deployment and for batching low-value redemptions under a single mandate where intent permits.

Broader impact. Autonomous agents that move money are a dual-use technology. The same x402 autonomy that enables machine commerce enables machine-speed theft if scope-lifting is unaddressed; an attacker who can inject one web page could, against an unbounded agent, drain allowances across a fleet. Our contribution is a brake: deterministic, on-chain bounds that fail safe (toward not paying) rather than fail open. But brakes can be misused—the same caveat machinery could enforce an operator’s policy against a user’s interest, and intent freezing concentrates trust in the approval-moment UI, which itself becomes a phishing target. Responsible deployment requires that mandates be human-readable at signing, revocable, and auditable, and that the anchor-

ing UI be hardened as carefully as the contracts. We open-source the extended AgentDojo benchmark and the enforcer contracts precisely so that these properties can be independently scrutinized.

8 Conclusion

We have argued that the defining security failure of agentic micropayments is not budget overrun but intent subversion: a prompt-injected agent spending the right amount for the wrong reason. Existing defenses bound magnitude (allowance caveats) or sacrifice autonomy (HITL gates); neither binds intent while preserving autonomy. Our intent-bound mandate chain freezes user intent into a signed EIP-712 mandate at the human-approval moment and enforces it through deterministic on-chain ERC-7710/7715 caveats that a corrupted agent cannot relax, backed by a CaMeL-style capability split that prevents untrusted data from originating payments. Across 5,400 injection cases per regime and a 400,000-trial stress round—with calibration values grounded in measured AgentDojo/InjecAgent rates and a $\pm 50\%$ sensitivity sweep confirming the conclusions do not depend on them—the regime drives scope-lift ASR from 49.7% to 0.0% while preserving 97.9% autonomous task completion at +27 ms and 72k gas, blocking replay/temporal attacks at 99.99%—meeting all nine pre-registered criteria.

We propose three concrete directions for future work. First, replace the Bernoulli simulation with a live end-to-end evaluation against real LLMs, a deployed Anvil/L2 chain, and the unmodified AgentDojo loop, to measure true ASR/FRR and to fuzz the Solidity enforcers and SD-JWT path for implementation flaws. Second, implement and harden the anchoring function Φ : build a benchmark of ambiguous, multi-step, and under-specified instructions and measure mandate fidelity, since Φ is the system’s principal residual risk and the source of the currently-assumed FRR/TCR@0 values. Third, extend intent binding from single payments to stateful intent—budgets that span sessions, conditional and streaming payments, and revocable delegations—while preserving the invariant that every redemption is the deterministic image of a human-signed mandate.

References

- [1] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, “Not What You’ve Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection,” in *Proc. ACM AISec*, 2023.
- [2] OWASP Foundation, “OWASP Top 10 for Large Language Model Applications,” 2023. [Online]. Available: <https://owasp.org/www-project-top-10-for-large-language-model-applications/>
- [3] A. Zou, Z. Wang, N. Carlini, M. Nasr, J. Z. Kolter, and M. Fredrikson, “Universal and Transferable Adversarial Attacks on Aligned Language Models,” *arXiv:2307.15043*, 2023.
- [4] K. Hines, G. Lopez, M. Hall, F. Zarfati, Y. Zunger, and E. Kiciman, “Defending Against Indirect Prompt Injection Attacks With Spotlighting,” *arXiv:2403.14720*, 2024.
- [5] S. Chen, J. Piet, C. Sitawarin, and D. Wagner, “StruQ: Defending Against Prompt Injection with Structured Queries,” in *Proc. USENIX Security*, 2025.
- [6] S. Willison, “The Dual LLM Pattern for Building AI Assistants That Can Resist Prompt Injection,” 2023. [Online]. Available: <https://simonwillison.net/2023/Apr/25/dual-llm-pattern/>
- [7] E. Debenedetti, I. Shumailov, T. Fan, J. Hayes, N. Carlini, D. Fabian, C. Kern, C. Shi, A. Terzis, and F. Tramèr, “Defeating Prompt Injections by Design (CaMeL),” *arXiv:2503.18813*, 2025.
- [8] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, and F. Tramèr, “AgentDojo: A Dynamic Environment to Evaluate Attacks and Defenses for LLM Agents,” in *Proc. NeurIPS Datasets and Benchmarks*, 2024.
- [9] Q. Zhan, Z. Liang, Z. Ying, and D. Kang, “InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents,” in *Findings of ACL*, 2024.
- [10] Google, “Agent Payments Protocol (AP2): An Open Protocol for Agent-Led Payments,” *Technical Specification*, 2025. [Online]. Available: <https://github.com/google-agentic-commerce/AP2>
- [11] M. Slabber et al., “ERC-7715: Grant Permissions from Wallets,” *Ethereum Improvement Proposals*, 2024. [Online]. Available: <https://eips.ethereum.org/EIPS/eip-7715>
- [12] R. Magnani et al., “ERC-7710: Smart Contract Delegation,” *Ethereum Improvement Proposals*, 2024. [Online]. Available: <https://eips.ethereum.org/EIPS/eip-7710>
- [13] V. Buterin, Y. Weiss, D. Tjiang, K. Gazso, N. Rai, and T. Wahrstätter, “ERC-4337: Account Abstraction Using Alt Mempool,” *Ethereum Improvement Proposals*, 2021.
- [14] R. Gosselin, “EIP-712: Typed Structured Data Hashing and Signing,” *Ethereum Improvement Proposals*, 2017.

- [15] P. Le and others, “EIP-3009: Transfer With Authorization,” Ethereum Improvement Proposals, 2020. [Online]. Available: <https://eips.ethereum.org/EIPS/eip-3009>
- [16] F. Vogelsteller and others, “EIP-1271: Standard Signature Validation Method for Contracts,” Ethereum Improvement Proposals, 2018.
- [17] D. Fett, K. Yasuda, and B. Campbell, “Selective Disclosure for JWTs (SD-JWT),” IETF Internet-Draft, 2024.
- [18] T. McLean, “Critical Vulnerabilities in JSON Web Token Libraries: Algorithm Confusion Attacks,” Auth0 Security Advisory, 2015.
- [19] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, “ReAct: Synergizing Reasoning and Acting in Language Models,” in Proc. ICLR, 2023.
- [20] Coinbase, “x402: An Open Protocol for Internet-Native Payments over HTTP 402,” Whitepaper, 2025. [Online]. Available: <https://www.x402.org>